

FUNCIONAMIENTO COMPLETO DE TRICKLE FEED BANCO - FLUJO REAL

Sistema de Procesamiento de Transacciones en Lotes - EPN CORE BANCARIO

Versión: 4.0 CORREGIDA (Flujo Real con TFFILE, TFBATFIN, tkin01)

Fecha: Mayo 16, 2026

Autor: Análisis Técnico Completo del Flujo Operativo

Clasificación: Documentación Técnica - Core Bancario

VISIÓN GENERAL DEL FLUJO REAL

Trickle Feed es un **motor de procesamiento completamente automatizado** que ejecuta un ciclo de 5 fases cada 30 segundos bajo supervisión de un demonio incansable. El flujo real involucra **5 programas COBOL principales**:

1. **TFFILE** (STEP 1) → Ingesta y loteo dinámico
 2. **TFTRCT** (Orquestador) → Máquina de estados
 3. **TFMX** (FASE 10) → Carga a réplica
 4. **TFBATFIN + tkin01** (FASE 20) → Procesamiento contable headless
 5. **XXXREP** (FASE 30) → Generación de reportes
-

1. ETAPA 1: TFFILE - INGESTA Y LOTEODINÁMICO (STEP 1)

¿Qué Espera TFFILE?

Entrada Requerida:

- Archivo plano **.TXT** en **BATCH-INPUT/** con nomenclatura exacta: **XXX-999-DDMMYY-XXXXXX-DDMMYY-00N.TXT**
- Ejemplo válido: **DEB-999-150526-140000-160526-001.TXT**
- Cada línea con formato delimitado por pipe: **OPERACION|CUENTA|MONTO**
 - **OPERACION**: DEB (débito), CRE (crédito), RET (retiro)
 - **CUENTA**: ID cliente (ej. 22345679)
 - **MONTO**: 15 dígitos, últimos 2 = decimales (ej. 000000000150002 = 1500.02)

¿Cómo Afecta TFFILE?

TFFILE PROCESA:

```
1. LEE PARAMETROS DE BD (TF_PARAMETROS)
  | MAX_REGISTROS_LOTE = 1000 (máximo por fragmento)
  | RUTA_INPUT = C:\banco\spool\Interfaces\BATCH-INPUT\
```

```

└─ RUTA_UPLOAD = C:\banco\spool\Interfaces\BATCH-UPLOAD-S\

2. LISTA Y VALIDA ARCHIVOS EN BATCH-INPUT/
└─ Busca: *.TXT
└─ Valida: Nomenclatura XXX-999-DDMMYY-XXXXXX-DDMMYY-00N.TXT
└─ Parsea: Tipo de operación (DEB, CRE, RET)
└─ Determina: TYPE_UPDATE (RRD000, RRC000, RRR000)

3. FRAGMENTA ARCHIVO SEGÚN MAX_REGISTROS_LOTE
└─ Abre archivo para lectura
└─ Para cada línea:
    └─ Cuenta registro
    └─ Si nuevo fragmento: Crea archivo en BATCH-UPLOAD-S/
    └─ Escribe línea
    └─ Si alcanzó límite: Cierra y prepara siguiente
└─ Ejemplo output:
    DEB-999-150526-140000-160526-001.TXT (1000 registros)
    DEB-999-150526-140000-160526-002.TXT (1000 registros)
    DEB-999-150526-140000-160526-003.TXT (500 registros)

4. INSERT EN TFFM (Tabla Maestra de Lotes)
└─ Por cada fragmento: INSERT INTO TFFM
    └─ FILE_NAME = nombre del fragmento
    └─ TYPE_UPDATE = RRD000/RRC000/RRR000
    └─ FASE = '00' (Nuevo - listo para procesarse)
    └─ ESTADO_REPLICA = 'R' (Ready)
    └─ FECHA_SISTEMA = 2026-05-16
    └─ PRIORITY = 5
    └─ COMMIT

5. **ACCIÓN CRÍTICA DE SEGURIDAD**
└─ MUEVE archivo original a BATCH-DONE/
└─ move /Y "...BATCH-INPUT\DEB-999..." "...BATCH-DONE\"
    └─ Evita reprocesamiento del mismo archivo

```

2. ETAPA 2: TFTRCT - ORQUESTACIÓN Y BALANCEO DE CARGA (STEP 2)

¿Qué Espera TFTRCT?

Condiciones Requeridas:

- Existencia de registros en TFFM con FASE < '40' (no completados)
- Al menos una réplica disponible: TF_REPLICAS.STATUS = 'L' (Libre)

¿Cómo Afecta TFTRCT?

TFTRCT = MÁQUINA DE ESTADOS REACTIVA

```

Busca: SELECT ... FROM TFFM WHERE FASE < '40' AND ESTADO_REPLICA = 'R'

Si encuentra lote:
├ Busca réplica disponible: SELECT ... FROM TF_REPLICAS WHERE STATUS='L'
├ Si hay réplica:
│   ├── Asigna: UPDATE TF_REPLICAS SET STATUS='O' (Ocupada)
│   ├── Vincula: UPDATE TFFM SET REPLICA_NO = 'TF0X'
│   └ Ejecuta máquina de estados según FASE:
│       ├── FASE='00' → Avanza a FASE='10'
│       ├── FASE='10' → Ejecuta TFMX
│       ├── FASE='20' → Ejecuta TFBATFIN + tkin01
│       ├── FASE='30' → Ejecuta XXXREP
│       └ FASE='40' → Cierra lote, libera réplica
└ Si NO hay réplica: Espera próximo ciclo

Loop nuevamente hasta que NO haya lotes con FASE < '40'

```

3. FASE 10: TFMX - CARGA ESPEJO POST-INICIALIZACIÓN

Entrada

- Archivo fragmentado en **BATCH-UPLOAD-S/*.TXT**
- Asignación de réplica (TF01-TF06)

Proceso

```

Lee archivo línea por línea:

Para cada línea (ej: "DEP|22345679|000000000150002"):
├ Parsea: OPERACION='DEP', CUENTA=22345679, MONTO=1500.02
├ INSERT en TF0X (réplica asignada):
│   ├── ID_LOTE = 100001
│   ├── ESTADO = 2 (Pendiente de procesar por Core)
│   ├── DATOS_TX = "DEP|22345679|000000000150002"
│   ├── REPLICA_NO = 'TF01'
│   ├── CUENTA = 22345679
│   ├── MONTO = 1500.02
│   └ OPERACION = 'DEP'
└ Siguiendo línea...

COMMIT: Todos los registros con ESTADO=2 (listos para procesamiento)

```

Salida

- Registros insertados en réplica (TF01-TF06) con **ESTADO=2**

- Tabla TFFM avanza: FASE='10' → '20'

4. FASE 20: TFBATFIN + tkin01 - EJECUCIÓN FINANCIERA HEADLESS

El Motor Dual

```

TFBATFIN (Motor Batch Headless)
↓
Abre cursor en réplica para registros con ESTADO=2
↓
Para cada registro:
├─ UPDATE ESTADO=3 (En proceso)
└─ CALL tkin01 (Motor Contable Puro)
    ↓
    tkin01 evalúa acción:
    ├── 'D' = DÉBITO
    ├── 'C' = CRÉDITO
    └─ 'R' = CONSULTA
        ↓
        [Afecta tabla CTACTES del Core Bancario]
        ↓
        Retorna: OK (0) o ERROR (código)
        ↓
    En TFBATFIN:
    ├── Si OK: UPDATE ESTADO=4 (OK)
    └─ Si ERROR: UPDATE ESTADO=7, COD_ERROR (Error)
  
```

Detalle de tkin01 - PROCESADOR DE DÉBITO

```

ACCIÓN 'D' (DÉBITO - Ej. Retiro)

1. SELECT SALDO_ACTUAL FROM ctactes WHERE ID_CLIENTE = 22345671
   → saldo_actual = 50.00

2. Verifica: saldo_actual (50.00) >= monto (100.00) ? NO X

3. ROLLBACK (sin cambiar ctactes)

4. Retorna: ERROR código=07 ("SALDO INSUFICIENTE PARA DEBITO")

En TFBATFIN:
  UPDATE TF01 SET ESTADO=7, COD_ERROR='E0023' WHERE ID_REGISTRO=9002
  
```

Detalle de tkin01 - PROCESADOR DE CRÉDITO

```

ACCIÓN 'C' (CRÉDITO - Ej. Depósito)
  
```

```
1. UPDATE ctactes
   SET SALDO_ACTUAL = SALDO_ACTUAL + 1500.02,
       COD_ULT_MOV = 7,                ← CÓDIGO 7: CRÉDITO BATCH TF
       IMPORTE_MOV = 1500.02,
       FECHA_HORA_ALT = NOW()         ← Timestamp servidor
   WHERE ID_CLIENTE = 22345679

2. COMMIT (Saldo afectado en Core Contable)

3. Retorna: OK (código=0)

En TFBATFIN:
   UPDATE TF01 SET ESTADO=4 WHERE ID_REGISTRO=9001
```

Tabla CTACTES - Impacto del Core Bancario

La tabla **CTACTES** es donde REALMENTE impacta el dinero:

ANTES (sin procesar):

ID_CLIENTE	SALDO_ACTUAL	COD_ULT_MOV	IMPORTE_MOV	FECHA_HORA_ALT
22345679	5000.00	0	0.00	2026-05-16 13:00:00
22345680	1000.00	0	0.00	2026-05-16 13:00:00
22345671	50.00	0	0.00	2026-05-16 13:00:00

DESPUÉS (procesamiento exitoso con Trickle Feed):

ID_CLIENTE	SALDO_ACTUAL	COD_ULT_MOV	IMPORTE_MOV	FECHA_HORA_ALT
22345679	3499.98	6	1500.02	2026-05-16 14:25:40
└ Débito aplicado (saldo: 5000 - 1500.02 = 3499.98)				
└ COD 6: DÉBITO BATCH TF				
22345680	1050.00	7	50.00	2026-05-16 14:25:45
└ Crédito aplicado (saldo: 1000 + 50 = 1050)				
└ COD 7: CRÉDITO BATCH TF				
22345671	50.00	0	0.00	2026-05-16 14:25:40
└ SIN CAMBIO (retiro rechazado por fondos insuficientes)				

Códigos de Operación en CTACTES

COD_ULT_MOV	Tipo	Descripción
6	DÉBITO	Débito procesado vía Batch Trickle Feed
7	CRÉDITO	Crédito procesado vía Batch Trickle Feed

5. FASE 30: XXXREP - REPORTE CONTABLE Y LIBERACIÓN

Entrada

- Registros procesados en réplica (ESTADO=4 o ESTADO=7)

Proceso

XXXREP:

1. Abre archivo de salida:
BATCH-UPLOAD-S/TRICKLE-FEED-REPORT/TF01_100001.OUT
2. Abre CURSOR en TF01:
SELECT DATOS_TX, ESTADO, COD_ERROR FROM TF01 WHERE ID_LOTE=100001
3. Para cada registro:
 - └ Si ESTADO=4 (OK): Escribe "DEP|22345679|000000000150002|OK|"
 - └ Si ESTADO=7 (Error): Escribe "RET|22345671|000000000100000|ERROR|E0023"
4. CLOSE archivo
5. COMMIT

Archivo generado (contenido de auditoría contable):

```
DEP|22345679|000000000150002|OK|
DEP|22345680|000000000005000|OK|
RET|22345671|000000000100000|ERROR|E0023
```

Salida

- Archivo **.OUT** en **BATCH-UPLOAD-S/TRICKLE-FEED-REPORT/**
- TFFM avanza: FASE='30' → '40'

6. FASE 40: CIERRE DEL LOTE

TFTRCT ejecuta cierre:

1. UPDATE TFFM SET FASE='40', ESTADO_REPLICA='C'
(Marca lote como COMPLETADO)
2. UPDATE TF_REPLICAS SET STATUS='L' WHERE REPLICA_NO='TF01'
(Libera réplica como disponible)
3. COMMIT FINAL

- ✓ LOTE 100001 COMPLETADO
 - └ Archivo original: BATCH-DONE/

- └ Archivo de salida: BATCH-UPLOAD-S/TRICKLE-FEED-REPORT/
- └ Registros procesados: 2 OK, 1 ERROR
- └ Impacto en Core Bancario (ctactes): 2 saldos actualizados

7. CICLO COMPLETO: 30 SEGUNDOS DESPUÉS

El demonio reinicia:

```
[T+30s] Demonio despierta

[TFFILE]
└ Busca nuevos archivos en BATCH-INPUT/
└ Si encuentra: Repite ETAPA 1
└ Si no hay: Termina

[TFDRMAIN → TFTRCT]
└ Busca lotes con FASE < '40'
└ Si encuentra: Repite máquina de estados
└ Si no hay: Termina (ciclo vacío)

[Espera 30 segundos] → goto inicio
```

8. IMPACTO OPERATIVO EN LAS TABLAS DEL CORE BANCARIO

8.1 Tabla TFFM (Cerebro Histórico del Loteamiento)

Evolución de un lote:

```
└ Registro 1 (ingresa)
└ ID_LOTE | FILE_NAME | FASE | ESTADO_REPLICA | REPLICA_NO | TOTAL_OK |
TOTAL_ERROR
└ 100001 | DEB-...-1 | '00' | 'R' | NULL | 0 | 0

└ Registro 2 (asignado a réplica)
└ 100001 | DEB-...-1 | '00' | 'R' | 'TF01' | 0 | 0

└ Registro 3 (cargado a réplica)
└ 100001 | DEB-...-1 | '10' | 'R' | 'TF01' | 0 | 0

└ Registro 4 (procesando)
└ 100001 | DEB-...-1 | '20' | 'R' | 'TF01' | 2 | 0
(parcial)

└ Registro 5 (generando reportes)
└ 100001 | DEB-...-1 | '30' | 'R' | 'TF01' | 2 | 1

└ Registro 6 (COMPLETADO)
```

```
100001 | DEB-...-1 | '40' | 'C' | 'TF01' | 2 | 1
└─ Lote terminado, réplica lista para nuevo lote
```

8.2 Tabla TF01-TF06 (Amortiguadores de Persistencia)

Registros en réplica (bitácora de auditoría):

ID_REGISTRO	ID_LOTE	ESTADO	DATOS_TX	OPERACION	CUENTA	MONTO	COD_ERROR
9001	100001	4	DEP ...	DEP	22345679	1500.02	NULL
9002	100001	7	RET ...	RET	22345671	100.00	E0023
9003	100001	4	DEP ...	DEP	22345680	50.00	NULL

Uso: Auditoría si cliente reclama "¿Me cobraron mal?"

└─ Se consulta TF01-TF06 para verificar si la transacción se procesó y su resultado

8.3 Tabla CTACTES (Impacto Real del Negocio)

Antes:

ID_CLIENTE	SALDO_ACTUAL	COD_ULT_MOV	FECHA_HORA_ALT
22345679	5000.00	0	2026-05-16 13:00:00

Después de Trickle Feed:

ID_CLIENTE	SALDO_ACTUAL	COD_ULT_MOV	IMPORTE_MOV	FECHA_HORA_ALT
22345679	3499.98	6	1500.02	2026-05-16 14:25:40

└─ Saldo cambió

└─ Código 6: Débito Batch TF

└─ Monto del movimiento

└─ Hora exacta (servidor)

9. GARANTÍAS DEL SISTEMA

✓ Automatización Completa

- Demonio ejecuta ciclo cada 30 segundos sin intervención
- Recuperación automática en case de errores
- No requiere supervisión manual constante

✓ Transaccionalidad

- COMMIT/ROLLBACK en puntos críticos
- Registros se marcan con ESTADO (2→3→4/7) para evitar duplicados
- Archivo original movido a BATCH-DONE para prevenir reprocesamiento

✓ Escalabilidad

- 6 réplicas paralelas (TF01-TF06)
- Load balancing simple (selecciona réplica más antigua)
- Capacidad de procesar múltiples lotes simultáneamente

✓ Auditoría y Trazabilidad

- TFFM: Historia completa del lote
- TF01-TF06: Bitácora detallada por registro
- CTACTES: Marca exacta de operación (COD_ULT_MOV=6/7) y timestamp
- BATCH-UPLOAD-S/TRICKLE-FEED-REPORT: Reporte contable final

Fin del Documento - Flujo Real Completo

Documentación Actualizada: Mayo 16, 2026